

awk

The Ultimate Data Extraction & Reporting Tool

Program Structure

```
awk [options] 'BEGIN { init } pattern { action } END { summary }' [FILE ... ]
```

BEGIN: Runs once before any lines are read (for setup).

pattern { action }: The core logic. Runs for each line matching the pattern. If pattern is omitted, action runs on every line.

END: Runs once after all lines are read (for reporting totals, averages, etc.).

Built-in Variables

\$0: The entire line.

NR: Number of Records (lines).

\$NF: The last field.

OFS: Output Field Separator.

\$1, \$2..: Fields 1, 2, etc.

NF: Number of Fields in line.

FS: Field Separator.

FILENAME: Current input file.

Common Options

-F fs: Set field separator.

-v var=val: Assign a variable.

-f file: Read program from a file.

Patterns

/regex/: Regex match on **\$0**.

\$1 ~ /err/: Field regex match.

NR > 1: Line number condition.

\$2 = "FAIL": Field comparison.

Control Flow & Actions

```
if (cond) { ... } else { ... }
```

```
for (i=1; i≤NF; i++) { ... }
```

```
while (cond) { ... }
```

print: Simple, space-separated output.

printf("fmt", vars): C-style formatted output.

String Functions

length(s)

tolower(s) / toupper(s)

substr(s, start, len)

sub(r, t, s): Substitute regex.

Associative Arrays

The core of `awk`'s power. Arrays use strings as keys, perfect for grouping and counting.

```
# Count occurrences of words in the first column
awk '{ count[$1]++ } END { for (k in count) print k, count[k] }'
```

